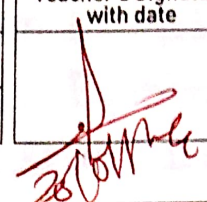


Name: Satvika Pansare Class: SOC 8 Batch: C
Subject: PI Roll No.: 55 Exp. No.: 1
Name of the Experiment: Introduction of
Python programming.
Performed on: _____
Submitted on: _____

Marks	Teacher's Signature with date
10	 26/05/2024

Q. 1. List the key feature and advantages of Python.
Also mention any two python IDEs or editors used for program development.

→ Key Feature of Python:

- 1] Simple and easy-to-learn syntax
- 2] Interpreted and high-level language
- 3] Object oriented and supports multiple paradigms.
- 4] Large standard library.

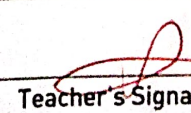
Advantages of Python:

- 1] Faster program development and debugging
- 2] Readable code and easy maintenance
- 3] Support many applications like web, data science, AI, etc.

Any two Python IDEs / Editors

- 1] PyCharm
- 2] IDLE

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐


Teacher's Signature

Q2. Explain the history and evolution of Python. Describe why Python has become popular for beginners and industry applications.

→ History and evolution of Python:

Python was created by Guido van Rossum in 1989 and released in 1991. It evolved from Python 2 to Python 3, which is faster, more secure, and widely used today.

Why Python is popular for Beginners:

- 1] easy & simple syntax, close to English
- 2] easy to learn and understand
- 3] less code compared to other languages.

Why Python is popular in Industry Applications:

- 1] Used in web development, data science, AI, ML & automation
- 2] Huge libraries and frameworks.
- 3] Cross-Platform and scalable

Q3. Write a python program to accept two numeric value from user and perform the following operations

- > Addition
- > Subtraction
- > Multiplication
- > Division

Display the result using proper input and

output prompting

$a = 3$

$b = 4$

$c = a + b$

`print(c)`

$a = 29$

$b = 13$

$c = a - b$

`print(c)`

$a = 24$

$b = 8$

$c = a * b$

`print(c)`

$a = 246$

$b = 28$

$c = a / b$

`print(c)`

Q4. Analyse the following Python code and identify the data types of each variable. Also explain output

$a = 10$

$b = 5.5$

$c = 3 + 4j$

$d = \text{True}$

`print(a+b)`

`print(a+b, type(c), d)`

→ Data types →

- a is an integer, type int
- b is a Floating-point number, type float.
- c is a complex num, type complex.
- d is a boolean value, type bool.

explanation of output →

1) $a+b$: The integer 10 is added to the float 5.5. Python automatically converts the integer to a float for the addition, resulting in a float value of 15.5

2) `type(c)`: This function returns the data type of the variable c, which is `<class 'complex'>`

3) d: This is the boolean value True

output of the code will be 15.5
`<class 'complex'>` True

5. Evaluate the use of relational, logical, and bitwise operators in python. Justify with suitable example where each type of operator is most appropriate.

→ 1. Relational (comparison) operators

Use: Relational operators are used to compare two values and return a Boolean result (True or False).

Common operators =

`>`, `<`, `>=`, `<=`, `==`, `!=`

`a = 10`

`b = 20`

`print(a < b)`

`print(a == b)`

checking eligibility (age, marks, salary)

They help program take decisions based on comparison

2). Logical Operators

Use: Logical operators are used to combine multiple conditions and return a Boolean result

Common operators:

and, or, not

`code =`

`age = 18`

`has_id = True`

`if age >= 18 and has_id:`

`print("Allowed to vote")`

Multiple conditions must be checked together
They make complex decision logic simple and readable.

3) Bitwise Operators

Use:

Bitwise operators work on individual bits of integers.

They are most appropriate in low-level programming, performance optimization, and hardware-related tasks.

Common operators:

$\&$, $|$, $\wedge$, $\vee$, $\ll$, $\gg$

Working with binary data

Flags, masks, & encryption.

They provide fast and direct control over data at the bit level.

Q6. Design and implement a menu-driven Python program that:

- > Accepts user input
- > Performs arithmetic, relational, logical, and assignment operations.
- > Display result clearly.

The program should handle int, float, and Boolean data type.

→ While True :

```
print("\n1. Arithmetic 2. Relational 3. Logical  
4. Assignment 5. Exit")
```

```
ch = int(input("Enter choice:"))
```

```
if ch == 1:
```

```
    a = float(input("Enter a: "))
```

```
    b = float(input("Enter b: "))
```

```
    print("Add:", a+b, "sub:", a-b, "mul:", a*b)
```

```
    print("Div:", a/b if b != 0 else "Not possible")
```

```
elif ch == 2:
```

```
    a = int(input("Enter a: "))
```

```
    b = int(input("Enter b: "))
```

```
    print(a > b, a < b, a == b, a != b)
```

```
elif ch == 3:
```

```
    x = bool(int(input("Enter x (1/0) : ")))
```

```
    y = bool(int(input("Enter y (1/0) : ")))
```

```
    print("AND:", x and y)
```

```
    print("OR:", x or y)
```

```
    print("NOT x:", not x)
```

```
elif ch == 4:
```

```
    a = int(input("Enter a: "))
```

```
    a += 2; print("a += 2: ", a)
```

```
    a *= 3; print("a *= 3: ", a)
```

```
elif ch == 5:
```

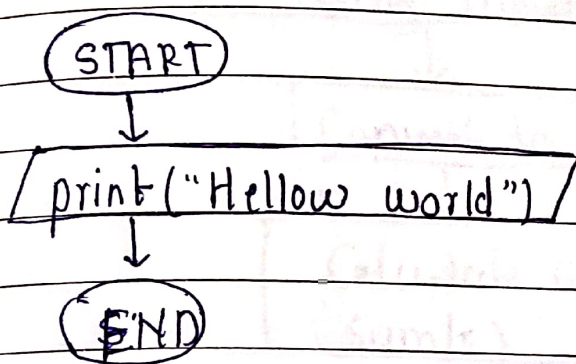
```
    break.
```

Q7 Write a program in python to display "Hello World".

code:-

```
print("Hello World").
```

Flow chart:-



Algorithm = 1. start

2. Display the message "Hello World"

3. Stop.

Q8. Write a program to perform average of Five numbers using input() function and type conversion concept.

Algorithm =

1. start

2. Input five numbers using input()

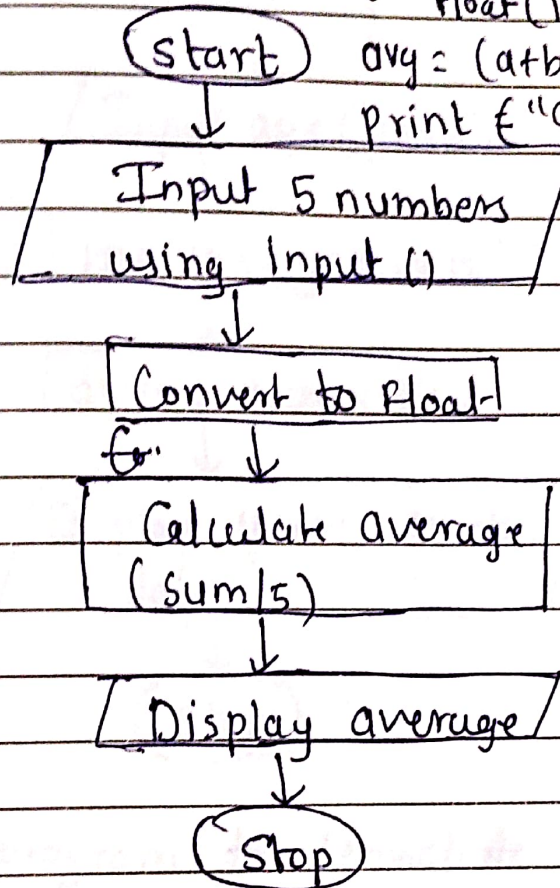
3. Convert inputs into float using type conversion

4. Add all five numbers

5. Divide the sum by 5 to get average.

6. Display the average
7. Stop

Flowchart =



Code =

```
a = Float(input("Avg of 1st no:"))  
b = Float(input("Avg of 2nd no:"))  
c = Float(input("Avg of 3rd no:"))  
d = Float(input("Avg of 4th no:"))  
e = Float(input("Avg of 5th no:"))  
avg = (a+b+c+d+e)/5  
print ("avg=", avg)
```

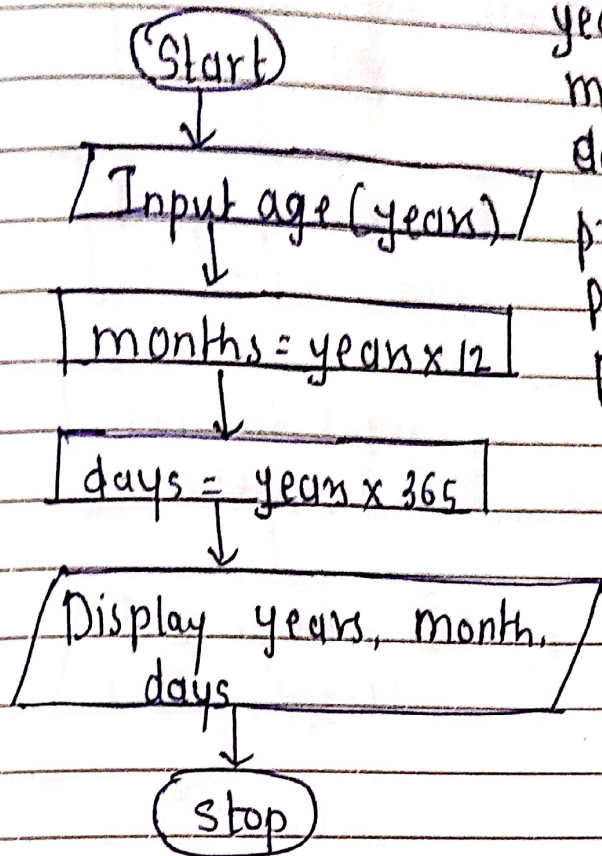
- 8.9. Write a program that prompts the user to enter their age and prints out their age in year, months, and days.

Algorithm =

1. Start
2. Ask the user to enter age in years
3. Convert age into months
 - months = years $\times$ 12
4. Convert age into days
 - days = years $\times$ 365

5. Display age in year, month and day
6. stop.

Flowchart =



Code =

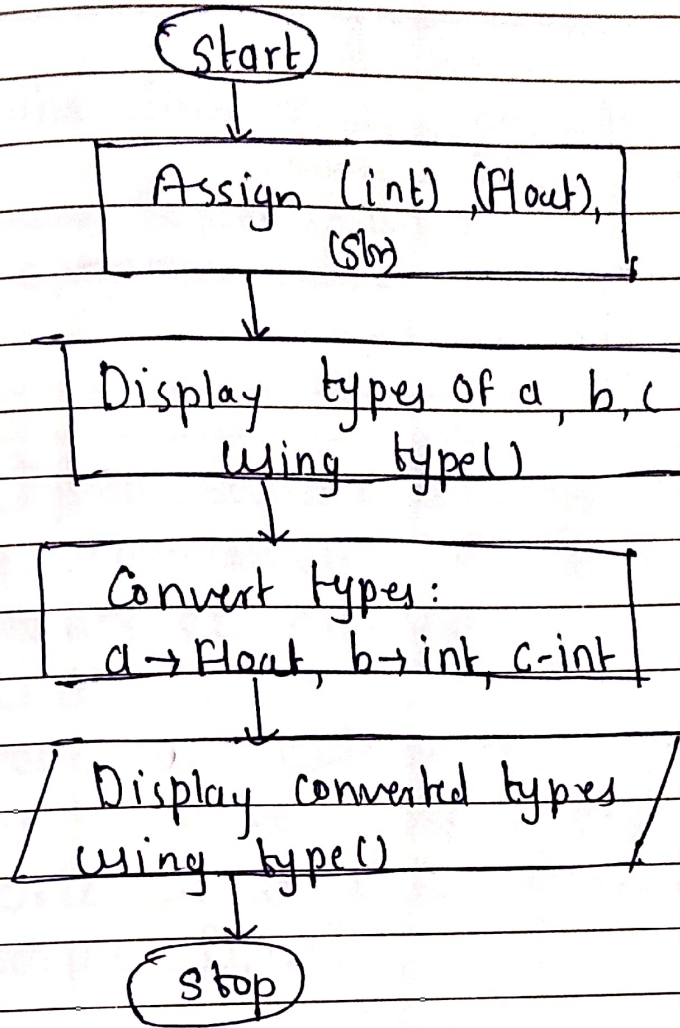
```
years = int(input())  
month = year * 12  
days = year * 365  
print("Age in years:", year)  
print("Age in months:", month)  
print("Age in days:", days)
```

10. Write a program to demonstrate the concept of data type & conversion.

Algorithm =

1. start
2. Assign value to variables a, b, c of types int, float, & string respectively
3. Display the original data type using type()
4. Convert a to float, b to int, c to int
5. Display the converted data types
6. stop.

Flowchart:



Code=

a=10

b=5.5

c="20"

print (type(a), type(b), type(c))

print (type(float(a)), type(int(b)), type(int(c)))



Name: Satvika Pansare
Subject: PL Class: SOC 8 Batch: C
Roll No.: 55
Name of the Experiment: Sequence-Types
and operator Exp. No.: 2
Performed on: _____
Submitted on: _____

Marks	Teacher's Signature with date
<u>10</u>	<u>[Signature]</u>

Q1. List the Python sequence data types and state one key characteristics of each with respect to mutability or ordering.

→ 1] List (list)

- Characteristic: Mutable and ordered
- Elements can be changed after creation, & the order of items is preserved.
- Example: `[1, 2, 3]`

2] Tuple (tuple)

- Characteristic: Immutable and ordered.
- Once created, elements cannot be modified, and the order is maintained.
- Example: `(1, 2, 3)`

3] String (str)

- Characteristic: Immutable and ordered
- Characters are stored in a fixed order and cannot be changed after creation.
- Example: `"Hello"`

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

[Signature]
Teacher's Signature

4) Range (range)

- characteristic: Immutable and ordered
- Represents a sequence of numbers in a specific order and cannot be modified.
- Example: range(1,10).

5) Bytes (bytes)

- Characteristic: Immutable and ordered.
- Stores an ordered sequence of bytes (integers 0-255) that cannot be changed
- Example: b"abc"

6) Bytearray (bytearray)

- characteristics: Mutable and ordered.
- similar to bytes, but elements can be modified after creation
- Example: bytearray(b"abc")

Q2 Explain how indexing and slicing work in python sequences. Illustrate how they differ when applied to strings and lists.

→ • Indexing and Slicing in Python Sequences:
Indexing is used to access a single element from a sequence using its position (index number).

- Index starts from 0 (left to right)
- Negative indexing starts from -1 (right to left).

Example :

```
s = "python"  
print(s[0])  
print(s[-1])
```

Slicing is used to extract a part (subsequence) of a sequence.

Syntax

sequence[start: stop: step]

Example :

```
s = "python"  
print(s[1:4])  
print(s[::2])
```

Difference Between String and List

Feature	String	List
Mutability	Immutable	Mutable
Indexing	Returns character	Return element
Slicing	Return new string	Return new list.

Example:

```
s = "Hello"
```

```
s[0] = "h"
```

Cannot change element.
↑
X Error (immutable)

```
l = [1, 2, 3]
```

```
l[0] = 10
```

✓ Allowed

3] Write a Python program that:

- > Uses indexing and slicing on a string or list, &

- > Apply at least two built-in functions to display meaningful results.

→ Python Program Using Indexing, Slicing & Built-in Function.

≠ using a string

```
text = "Programming"
```

```
# Indexing
```

```
first_char = text[0]
```

```
last_char = text[-1]
```

```
# slicing
```

```
middle_part = text[3:8]
```

```
# Built in function
```

```
length = len(text)
```

```
uppercase_text = text.upper()
```

```
print("Original Text:", text)
```

```
print("First Character:", first_char)
```

```
print("Last Character:", last_char)
```

```
print("Middle Part:", middle_part)
```



```
print("Length of Text:", length)  
print("Uppercase Text:", uppercase_text)
```

Built-in Function used:

- len() → Finds length
- upper() → Convert to uppercase

Q4] Analyse the difference between equality (==) and identity (is) when used with sequence types. Discuss how object mutability and memory reference affect program behaviour.

→ Difference Between Equality (==) and Identity (is)

Operator

Meaning

==

Checks if values are equal

is

Checks if two variables refer to the same object in memory.

Example :

a = [1, 2, 3]

b = [1, 2, 3]

c = a

print(a == b)

True

print(a is b)

False (different memory locations)

print(a is c)

True

- ### Role of Mutability and memory reference
- Mutable Objects (like list) can be changed after creation. If two variable refer to the same list, changes affect both.
 - Immutable Objects (like strings) cannot be modified. Any change creates a new object in memory.

Example:

```
x = [10, 20]
```

```
y = x
```

```
y.append(30)
```

```
print(x)
```

```
# [10, 20, 30]
```

Thus:

- == Compares Content
- is Compares memory address
- Mutability affects how reference behave in programs.

Q5] Evaluate the suitability of lists, sets, and dictionaries for storing and managing unique student records. Justify which data structure is most efficient and why.

→
~~suitability~~ suitability of Lists, sets, and Dictionaries for student records

Data structure

Advantages

Disadvantages

List

Ordered, easy to
store multiple
records

Allows duplicates,
Slow search

set

Stores only unique
values

No indexing, cannot
store complex
structure record
easily.

Dictionary

stores key-value
pairs, fast access

key must be
unique.

- List are not efficient for unique student records because duplicates can occur and searching is slow. (O(n)).
- Sets ensure uniqueness but are not suitable for structured data like student name, roll number, marks.
- Dictionaries are most efficient because:
 - ✓ each student can have a unique key
 - ✓ fast lookup (O(1) Average time)
 - ✓ can store structured data as values.

Example.

Students = {

101: { "name": "Asha", "marks": 85 }

102: { "name": "Ravi", "marks": 90 }

}

Q6] Design and implement a Python program that uses:

- > Appropriate Sequence types
- > Indexing or Slicing
- > Built-in function

to manage and process student data efficiently.

→ manage & process Student Data.

This program uses:

- > sequence type → List and Dictionary
- > Indexing & Slicing
- > Built-in functions → len(), max(), min(), sum(), sorted()

Code →

```
students = [  
    {"name": "Asha", "marks": [85, 78, 92]},  
    {"name": "Ravi", "marks": [88, 76, 95]},  
    {"name": "Neha", "marks": [90, 82, 89]}  
]
```

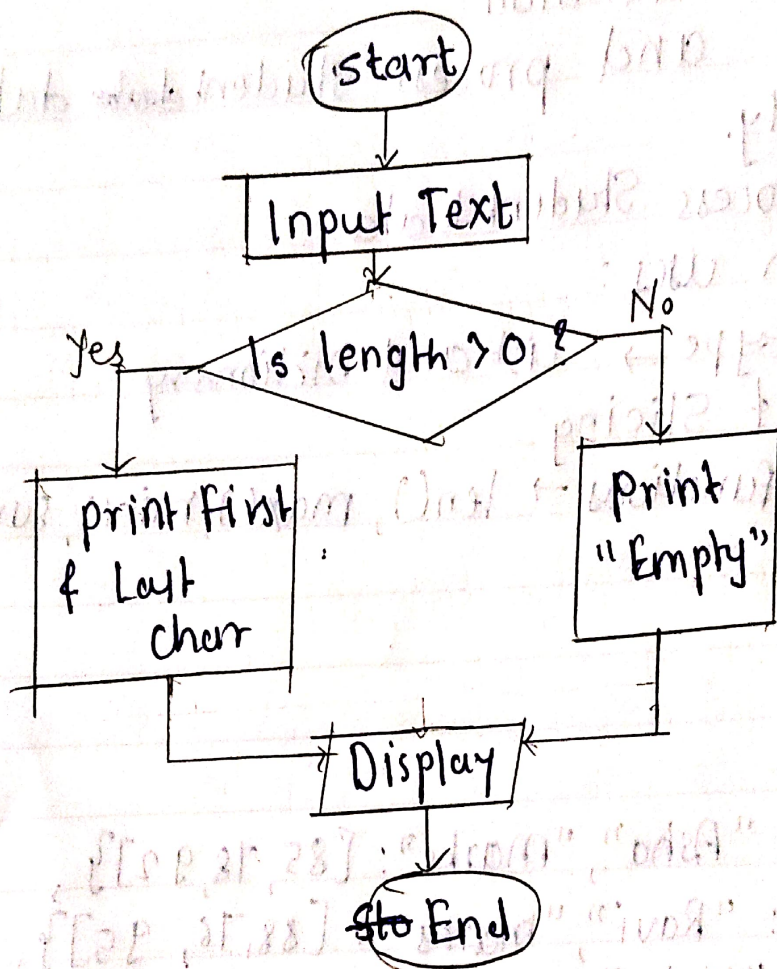
```
print("Total Students:", len(students))  
print("First Student:", students[0]["name"])  
print("First Two Student:", [s["name"] for s in  
    student[:2]])
```

for s in students:

total = sum(s["marks"])

avg = total / len(s["marks"])

print(s["name"], "→ Avg:", round(avg, 2))




```
ranked = sorted(students, key = lambda x:  
sum(x["marks"])/len(x["marks"]), reverse=True)  
print("Topper : ", ranked[0]["name"])
```

Uses :

- > list & Dictionary
- > Indexing & slicing
- > Built-in funⁿ → len(), sum(), sorted(), round()

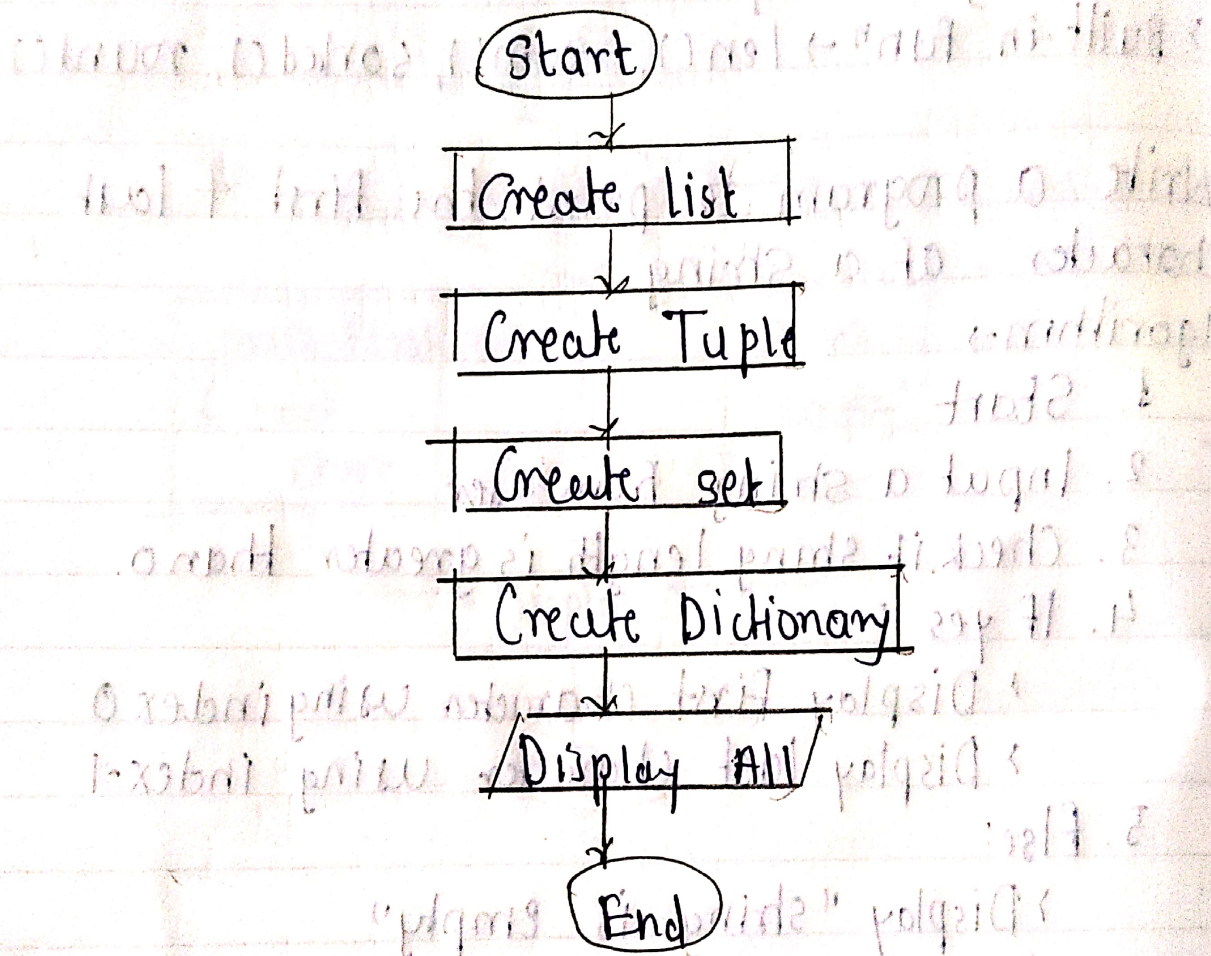
Q1] Write a program to print the first & last character of a string.

→ Algorithm →

1. Start
2. Input a string from user
3. Check if string length is greater than 0.
4. If yes :
 - > Display first character using index 0
 - > Display last character using index -1
5. Else:
 - > Display "string is empty"
6. Stop.

Code →

```
text = input("Enter a string:")  
if len(text) > 0:  
    print("First character:", text[0])  
    print("Last character:", text[-1])  
else:  
    print("string is empty.")
```

Q8] Write a program to create a list, tuple, set and dictionary.

→ Algorithm →

1. Start
2. Create a list with some elements
3. Create a tuple with some elements
4. Create a set with some elements
5. Create a dictionary with key-value pairs.
6. Display all data structures
7. Stop

Code →

```
my_list = [10, 20, 30, 40]
```

```
my_tuple = (1, 2, 3, 4)
```

```
my_set = {5, 6, 7, 8}
```

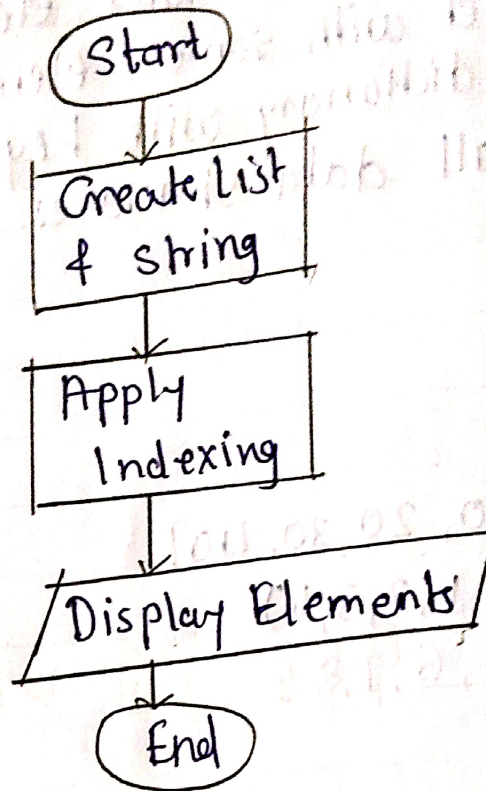
```
my_dict = {  
    "name": "Asha",  
    "age": 20,  
    "course": "BCA"  
}
```

```
print("List:", my_list)
```

```
print("Tuple:", my_tuple)
```

```
print("Set:", my_set)
```

```
print("Dictionary:", my_dict)
```

Q9] Write a program to understand the concept of indexing.

→ Algorithm →

1. Start

2. Create a list and a string

3. Access elements using positive indexing.

4. Access elements using negative indexing.

5. Display the results

6. Stop.

Code →

```
numbers = [10, 20, 30, 40, 50]
```

```
text = "Python"
```

```
print("First element in list:", numbers[0])
```

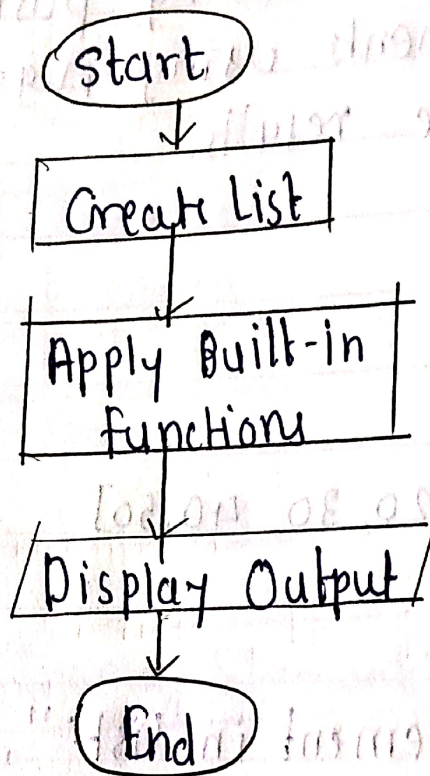
```
print("Third element in list:", numbers[2])
```

```
print("Last element in list:", numbers[-1])
```

```
print("First character in string:", text[0])
```

```
print("Fourth character in string:", text[3])
```

```
print("Last character in string:", text[-1])
```

Q10] Write a program to understand the concept of built-in function.

→ Algorithm →

1. Start
2. Create a list of number.
3. Apply built-in functions (len, max, min, sum)
4. Display the results.
5. Stop

Code →

```
numbers = [10, 25, 5, 40, 15]
print("List:", numbers)
print("Length of list:", len(numbers))
print("Maximum value:", max(numbers))
print("Minimum value:", min(numbers))
print("Sum of elements:", sum(numbers))
```